Data Science Cheat Sheet

Python ≥ 3.12 · NumPy 2.5 · pandas 3.1 · scikit-learn 1.9
Checked October 2026 · albertoirurueta.github.io/docs

Workflow (CRISP-DM)

Business → Data understanding → Preparation → Modeling → Evaluation → Deployment

Business	decision, users, success in numbers, constraints (privacy, time)
Understand	load, shapes, types, ranges, missing values, plots
Prepare	clean, join, derive features, one modeling table
Model	pick candidates, decide validation *before* fitting
Evaluate	judge against the business goal; review for leakage
Deploy	report or service, monitoring, retraining, lessons

The loop is iterative: findings in any phase send you back to an earlier one.

Environments

venv

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pip freeze > requirements.txt
```

uv

```
uv init --python 3.13 proj
uv add numpy pandas scikit-learn
uv add --dev jupyterlab ipykernel
uv run python script.py     # lock + sync
uv sync                     # rebuild from uv.lock
```

conda

```
conda create -n proj python=3.13 pandas scikit-learn
conda activate proj
conda env export --from-history > environment.yml
```

Use Miniforge and `conda-forge` only; do not mix `defaults` with it. Prefer conda for the base stack, pip last. **kernel** `python -m ipykernel install --user --name proj`

Jupyter shortcuts and magics

Key	Action
`Shift+Enter`	run cell, select next
`Ctrl+Enter` / `Alt+Enter`	run in place / run, insert below
`Esc` / `Enter`	command mode / edit mode
`A` / `B`	insert cell above / below (command mode)
`D D`	delete cell
`M` / `Y`	cell to Markdown / code
`Tab` / `Shift+Tab`	complete / show signature
`%timeit expr`	time a line; `%%timeit` a cell
`%load_ext autoreload` `%autoreload 2`	re-import edited modules
`%whos` · `%run f.py`	variables · run a script
`%debug`	post-mortem debugger
`%pip install x` · `!cmd`	installs into the kernel's environment, unrecorded

Run top to bottom from a fresh kernel before sharing.

EDA checklist

- **Shape and types:** `df.shape`, `df.info()`, `df.head()`, `df.sample(5)`
- **Summaries:** `df.describe(include="all")`
- **Quality:** `df.isna().mean()`, `df.duplicated().sum()`, impossible values, units, ranges
- **Univariate:** histogram or ECDF, box plot, `value_counts(normalize=True)`
- **Outliers:** IQR fences Q1−1.5·IQR and Q3+1.5·IQR; investigate, do not delete blindly
- **Bivariate:** scatter, grouped box plots, cross-tabs, `df.corr(numeric_only=True)`
- **Target:** class balance, distribution, link to each feature
- **Time:** range, gaps, trend, seasonality
- Summary statistics can hide structure (Anscombe's quartet): always plot.
- Correlation is not causation.

Statistical test chooser (scipy.stats)

Question	Test
2 independent means	`ttest_ind(equal_var=False)` (Welch)
2 paired samples	`ttest_rel`; `wilcoxon` if not normal
2 groups, no normality	`mannwhitneyu`
3+ groups	`f_oneway`; `kruskal` if not normal
2 categoricals	`chi2_contingency` (expected ≥ 5 per cell); `fisher_exact` for small 2x2
One proportion	`binomtest(k, n, p)`
Association	`pearsonr` (linear); `spearmanr` (rank)
Any statistic, few assumptions	`bootstrap`, `permutation_test`

Match the test to the design (paired or independent). Pick one- or two-sided *before* looking. Report effect size and a confidence interval, not only *p*. Many tests: correct with `statsmodels...multipletests` (Bonferroni, `fdr_bh`).

Chart chooser

Distribution	histogram, ECDF, box, violin
Relationship	scatter (`hexbin` when dense), pairplot
Over time	line
Compare categories	bar from zero; sorted horizontal bars for long labels
Composition	stacked bar
Matrix of values	heatmap (diverging map for correlations)

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(x, bins=30); ax.set_xlabel("x")
fig.savefig("fig.png", dpi=150)
```

Explicit Figure and Axes interface; axes-level seaborn functions take `ax=`, figure-level (`relplot`, `displot`, `catplot`) do not. Use `viridis`, never `jet`; stay colour-blind safe.

Metric chooser

Task	Metric (scoring string)
Balanced classes	accuracy
Imbalanced	`average_precision`, `f1`, `balanced_accuracy`
Misses costly	recall; false alarms costly: precision
Ranking quality	`roc_auc`
Probabilities	`neg_log_loss`, `neg_brier_score`
Regression	`neg_mean_absolute_error` (robust), `neg_root_mean_squared_error` (big errors), `r2`
Clustering	`silhouette_score`

A model that always predicts the majority class can score 0.965 accuracy at 3.5% positives. Compare with a baseline (`DummyClassifier`).

Leakage rules

- **Split first**, then fit anything that learns from data (scalers, imputers, encoders, selectors) on the training part only.
- Put preprocessing and model in one `Pipeline` and cross-validate the pipeline.
- Repeated entities: `GroupKFold`. Time-ordered data: `TimeSeriesSplit`. Imbalance: `StratifiedKFold`.
- No feature built from the target or from the future.
- Tune on validation or CV; touch the test set once.
- Duplicates across train and test inflate scores: deduplicate before splitting.

```
split train / validation / test
 fit(train) → tune(CV) → score(test) once
```

Reproducibility rules

1. One command from raw data to results.
2. Code in Git; results name their commit.
3. Raw data never edited; version it (DVC) or hash it.
4. Lock file committed, Python version recorded.
5. Seeds explicit: `np.random.default_rng(0)`, `random_state=0`; report variation across seeds.
6. Parameters in a config file; experiments tracked (MLflow).
7. Notebooks run top to bottom; strip outputs or pair with scripts.
8. README to rebuild the environment; test from a clean checkout.

Versions and ethics

Python	≥ 3.12 (examples on 3.13)
NumPy	2.5 (NumPy 2 renames apply)
pandas	3.1: Copy-on-Write on, `str` dtype default
scikit-learn	1.9

Minimise and protect personal data; check consent and licence; test for bias across groups (sampling, label, measurement); document data and model limits.